Lab 5 - Implement a Dynamic String List Using struct and Recursion

Total points: 50

Learning Objectives

- Define a `struct` with private and public members
- Use constructor and destructor
- Implement dynamic memory allocation
- Create private helper methods
- Handle exceptions
- Print data using member functions
- Use recursion in programming

Part 1 – Preparation for Exceptions

Question 1 – Review the quiz for chapter 16 using Introduction to Programming with C++, Third Edition. Get the screenshot from the final answer and paste it into **answers.pdf**.

Part 2 – Analysis and Debugging

Question 2 – [20 points] Each of the following declarations, programs, and program segments has errors. Locate as many as you can. (**answers.pdf**)

1.	`Struct {`
	`  int x;`
	`  float y;`
	`};`

2.	`struct Values {`
	`  string name;`
	`  int age;`
	`}`

3.	`struct TwoVals {`
	`  int a, b;`
	`};`
	`int main () {`
	`  TwoVals.a = 10;`
	`  TwoVals.b = 20;`
	`  return 0;`
	`}`

4.	`#include <iostream>`
	`using namespace std;`
	`struct ThreeVals {`
	`  int a, b, c;`
	`};`
	`int main() {`
	`  ThreeVals vals = {1, 2, 3};`
	`  cout << vals << endl;`
	`  return 0;`
	`}`

5.

```
#include <iostream>
#include <string>
using namespace std;
struct names {
	string first;
	string last;
};
int main (){
	names customer = "Smith", "Orley";
	cout << names.first << endl;
	cout << names.last << endl;
	return 0;
}
```

6.

```
struct FourVals {
	int a, b, c, d;
};
int main () {
	FourVals nums = {1, 2, , 4};
	return 0;
}
```

7.

```
#include <iostream>
using namespace std;
struct TwoVals
{
	int a = 5;
	int b = 10;
};
int main(){
	TwoVals v;
	cout << v.a << " " << v.b;
	return 0;
}
```

8.

```
struct TwoVals {
	int a = 5;
	int b = 10;
};
int main() {
	TwoVals varray[10];
	varray.a[0] = 1;
	return 0;
}
```

9.

```
struct TwoVals {
	int a;
	int b;
};
TwoVals getVals() {
	TwoVals.a = TwoVals.b = 0;
}
```

10.

```
struct ThreeVals {
	int a, b, c;
};
int main () {
```

	TwoVals s, *sptr = nullptr;
	sptr = &s;
	*sptr.a = 1;
	return 0;
	}

Part 2 - Programming

Question 3 – Create a struct called **StringList** that stores a dynamic list of strings. The list should:

- Start with a capacity of 3.
- Automatically resize when full.
- Allow adding new strings and retrieving strings by index.
- Throw an exception if an invalid index is accessed.

Tasks

Define the struct StringList:

- Private members:
  - string* items (pointer to dynamic array)
  - int size (current number of elements)
  - int capacity (maximum capacity)
  - A private method resize() that increases capacity by 2.
- Public members:
  - Constructor: initializes size = 0, capacity = 3, allocates memory.
  - Destructor: frees allocated memory.
  - Method add(string) to add a new string.
  - Method get(int) to return a string at a given index (throw out_of_range if invalid).
  - Method print() to display all strings.

A. **[25 points]** To implement above tasks, download **StringList.cpp** and complete the program. Use the main function to test your design. Do not change the main function, except for uncommenting the line for testing exception.

B. **[5 points]** Calculate the time-complexity of every member function in the struct. Write your final answers as an inline comment above each member function.

Submission

Submit a zip file including the completed file **StringList.cpp** and **answers.pdf** in the Brightspace before the due date.